

JSON Server Deploy Guide for Next.js Projects

- [JSON Server Deploy Guide for Next.js Projects](#)
 - [Bangla Detailed Version](#)
 - [আপনি এই গাইড থেকে কী শিখবেন?](#)
 - [গুরুত্বপূর্ণ Version Note](#)
 - [Prerequisites](#)
- [1. Project Folder Structure তৈরি করুন](#)
- [2. Project Initialize করুন](#)
- [3. JSON Server Install করুন](#)
- [4. db.json তৈরি করুন](#)
- [5. package.json Configure করুন](#)
 - [Script ব্যাখ্যা](#)
- [6. public Folder তৈরি করুন](#)
- [7. Local এ Test করুন](#)
- [8. API Route Test Examples](#)
 - [সব products দেখতে](#)
 - [নির্দিষ্ট product দেখতে](#)
 - [সব books দেখতে](#)
 - [Query দিয়ে filter করতে](#)
 - [Sort করতে](#)
 - [Search করতে](#)
- [9. GitHub-এ Push করুন](#)
- [10. Render-এ Deploy করুন](#)
 - [Render Steps](#)
 - [Important Render Settings](#)
- [11. Live API URL](#)
- [12. Render Free Plan Note](#)
- [13. গুরুত্বপূর্ণ Data Persistence Warning](#)
- [14. Next.js Project থেকে API Call](#)
 - [Option A: Environment Variable ব্যবহার করুন](#)
- [15. Next.js App Router Example](#)
 - [Code ব্যাখ্যা](#)

- [16. Client Component থেকে Fetch Example](#)
- [17. Common Errors & Fixes](#)
- [18. Final Checklist](#)
- [19. Best Practices](#)
 - [Short Summary](#)

JSON Server Deploy Guide for Next.js Projects

Bangla Detailed Version

এই গাইডে আমরা JSON Server দিয়ে খুব সহজে একটি fake REST API তৈরি করব, তারপর সেটি Render-এ deploy করব, এবং শেষে Next.js project থেকে সেই live API fetch করব।

এটি মূলত practice project, prototype, assignment, frontend development বা backend ছাড়া API ব্যবহার করার জন্য খুব helpful।

আপনি এই গাইড থেকে কী শিখবেন?

এই গাইড শেষ করলে আপনি পারবেন:

- db.json দিয়ে fake database তৈরি করতে
 - JSON Server দিয়ে REST API চালাতে
 - Local machine-এ API test করতে
 - GitHub-এ project push করতে
 - Render-এ API deploy করতে
 - Next.js App Router থেকে live API fetch করতে
 - .env.local ব্যবহার করে API URL manage করতে
 - Common deployment errors fix করতে
-

গুরুত্বপূর্ণ Version Note

এই guide-এ beginner-friendly এবং stable setup দেখানোর জন্য json-server@0.17.4 ব্যবহার করা হয়েছে।

কারণ json-server -এর newer 1.x version এখনো beta হতে পারে। Beta version-এ command, routing বা behavior change হওয়ার chance থাকে। তাই learning এবং deployment practice-এর জন্য stable version ব্যবহার করাই safer।

Prerequisites

শুরু করার আগে আপনার machine-এ এগুলো থাকতে হবে:

- Node.js installed
- npm installed
- Git installed
- GitHub account
- Render account
- Basic JavaScript / Next.js knowledge

Check করতে পারেন:

```
node -v
npm -v
git --version
```

1. Project Folder Structure তৈরি করুন

প্রথমে project structure এমন হবে:

```
my-json-api/
├── db.json
├── package.json
├── public/
└── .gitkeep
```

Folder গুলোর কাজ

File / Folder	কাজ
db.json	এটিই fake database হিসেবে কাজ করবে
package.json	project scripts এবং dependencies রাখবে
public/	static files রাখার folder; empty folder GitHub-এ রাখার জন্য .gitkeep ব্যবহার করা হয়

Note: public/ folder না রাখলেও অনেক সময় server চলতে পারে, কিন্তু deploy/debugging সহজ করার জন্য এবং static folder related issue এড়াতে এটি রাখা ভালো।

2. Project Initialize করুন

Terminal খুলে নিচের command দিন:

```
mkdir my-json-api
cd my-json-api
npm init -y
```

ব্যাখ্যা

- `mkdir my-json-api` → নতুন folder তৈরি করে
- `cd my-json-api` → folder-এর ভিতরে যায়
- `npm init -y` → default `package.json` তৈরি করে

3. JSON Server Install করুন

Stable version install করুন:

```
npm install json-server@0.17.4
```

কেন version specify করা হচ্ছে?

```
npm install json-server
```

এই command দিলে latest version install হবে। কিন্তু latest version beta হলে deploy বা command-related issue হতে পারে। তাই fixed stable version ব্যবহার করলে project predictable থাকে।

4. db.json তৈরি করুন

Project root folder-এ `db.json` নামে একটি file তৈরি করুন:

```
{
  "products": [
    {
      "id": 1,
      "name": "Wireless Mouse",
      "price": 29.99,
      "category": "Electronics",
      "stock": 50,
      "rating": 4.5
    },
    {
```

```

    "id": 2,
    "name": "Mechanical Keyboard",
    "price": 79.99,
    "category": "Electronics",
    "stock": 30,
    "rating": 4.7
  },
  ],
  "books": [
    {
      "id": 1,
      "title": "Atomic Habits",
      "author": "James Clear",
      "price": 18.99,
      "rating": 4.8
    }
  ]
}

```

এটা কীভাবে API বানায়?

JSON Server db.json -এর top-level key দেখে route তৈরি করে।

এই example-এ আছে:

```

"products": []
"books": []

```

তাই automatically endpoints হবে:

```

/products
/products/1
/books
/books/1

```

5. package.json Configure করুন

আপনার package.json file-এর scripts এবং dependencies অংশ এমন করুন:

```

{
  "name": "my-json-api",
  "version": "1.0.0",
  "scripts": {
    "start": "json-server --watch db.json --host 0.0.0.0 --port $PORT",
    "dev": "json-server --watch db.json --port 4000"
  },
  "dependencies": {
    "json-server": "0.17.4"
  }
}

```

Script ব্যাখ্যা

Local development script

```
npm run dev
```

এটি local machine-এ server চালাবে:

```
http://localhost:4000
```

Production / Render script

```
npm start
```

Render এই command run করবে।

```
--host 0.0.0.0
```

এর মানে server শুধু নিজের computer-এর ভিতরে না থেকে বাইরে থেকেও request নিতে পারবে। Deploy platform-এ এটা দরকার।

```
--port $PORT
```

Render নিজে একটি port assign করে। তাই fixed port যেমন 4000 দিলে deploy fail হতে পারে। \$PORT ব্যবহার করলে Render যেই port দেয়, server সেটাতেই চলে।

6. public Folder তৈরি করুন

Root folder-এ public folder তৈরি করুন:

```
mkdir public
```

তারপর public folder-এর ভিতরে .gitkeep file তৈরি করুন:

```
touch public/.gitkeep
```

Windows PowerShell হলে:

```
New-Item -ItemType File public/.gitkeep
```

কেন .gitkeep ?

Git সাধারণত empty folder track করে না। তাই folder-এর ভিতরে .gitkeep রাখলে GitHub-এ public/ folder থাকবে।

7. Local এ Test করুন

প্রথমে dependency install করুন:

```
npm install
```

তারপর server চালান:

```
npm run dev
```

Browser-এ visit করুন:

```
http://localhost:4000/products  
http://localhost:4000/books  
http://localhost:4000/products/1
```

যদি JSON data দেখা যায়, তাহলে local setup successful।

8. API Route Test Examples

সব products দেখতে

```
GET /products
```

নির্দিষ্ট product দেখতে

```
GET /products/1
```

সব books দেখতে

```
GET /books
```

Query দিয়ে filter করতে

```
GET /products?category=Electronics
```

Sort করতে

```
GET /products?_sort=price&_order=asc
```

Search করতে

```
GET /products?q=mouse
```

9. GitHub-এ Push করুন

প্রথমে Git initialize করুন:

```
git init
git add .
git commit -m "initial: json server setup"
```

Branch name `main` করে নিন:

```
git branch -M main
```

GitHub-এ একটি new public repository তৈরি করুন। তারপর remote add করুন:

```
git remote add origin https://github.com/username/my-json-api.git
git push -u origin main
```

username অংশে আপনার GitHub username বসাবেন।

10. Render-এ Deploy করুন

Render-এ JSON Server deploy করার জন্য Web Service ব্যবহার করবেন, Static Site না।

কারণ JSON Server একটি running server/API। Static Site শুধু HTML/CSS/JS serve করার জন্য।

Render Steps

1. render.com এ login করুন
2. Dashboard থেকে New এ click করুন
3. Web Service select করুন
4. GitHub repository connect করুন
5. আপনার JSON Server repo select করুন
6. নিচের settings দিন

Important Render Settings

Setting	Value
Runtime / Language	Node
Build Command	npm install
Start Command	npm start
Root Directory	blank রাখুন, যদি repo root-এই project থাকে
Instance Type	Free / Starter যেটা চান
Port	manually set করার দরকার নেই

তারপর Create Web Service বা Deploy button চাপুন।

11. Live API URL

Deploy complete হলে Render আপনাকে একটি URL দেবে:

```
https://your-app-name.onrender.com
```

এখন API endpoints হবে:

```
https://your-app-name.onrender.com/products  
https://your-app-name.onrender.com/books  
https://your-app-name.onrender.com/products/1
```

Browser-এ open করে test করুন।

12. Render Free Plan Note

Render free web service কিছুক্ষণ inactive থাকলে sleep/spin down হতে পারে। তাই অনেক সময় প্রথম request একটু slow হতে পারে। কিছুক্ষণ পর API response আসবে।

13. গুরুত্বপূর্ণ Data Persistence Warning

JSON Server real production database না। এটি mainly mock API / practice API।

Render-এ deploy করার পর যদি আপনি POST, PUT, PATCH, DELETE করেন, data temporarily change হতে পারে। কিন্তু redeploy/restart হলে file system reset হতে পারে, তাই changes হারিয়ে যেতে পারে।

Production project হলে real database ব্যবহার করুন, যেমন:

- MongoDB
 - PostgreSQL
 - Supabase
 - Firebase
 - Render Postgres
-

14. Next.js Project থেকে API Call

Option A: Environment Variable ব্যবহার করুন

Next.js project-এর root folder-এ `.env.local` file তৈরি করুন:

```
NEXT_PUBLIC_API_URL=https://your-app-name.onrender.com
```

কেন `.env.local` ?

API URL code-এর মধ্যে hardcode করলে পরে URL change হলে অনেক file update করতে হয়। Environment variable ব্যবহার করলে এক জায়গা থেকেই URL manage করা যায়।

15. Next.js App Router Example

`app/page.jsx`:

```

async function getProducts() {
  const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/products`, {
    next: { revalidate: 60 }
  });

  if (!res.ok) {
    throw new Error("Failed to fetch products");
  }

  return res.json();
}

export default async function Home() {
  const products = await getProducts();

  return (
    <main>
      <h1>Products</h1>

      <ul>
        {products.map((product) => (
          <li key={product.id}>
            {product.name} - ${product.price}
          </li>
        ))}
      </ul>
    </main>
  );
}

```

Code ব্যাখ্যা

```
fetch(`${process.env.NEXT_PUBLIC_API_URL}/products`)
```

এখানে API URL `.env.local` থেকে আসছে।

```
next: { revalidate: 60 }
```

এর মানে Next.js 60 seconds cache রাখবে। 60 seconds পরে নতুন request করলে fresh data check করবে।

```
if (!res.ok)
```

API response successful না হলে error throw করবে।

16. Client Component থেকে Fetch Example

যদি আপনি browser-side থেকে fetch করতে চান:

```

"use client";

import { useEffect, useState } from "react";

export default function ProductsPage() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/products`)
      .then((res) => res.json())
      .then((data) => setProducts(data));
  }, []);

  return (
    <div>
      <h1>Products</h1>

      {products.map((product) => (
        <p key={product.id}>
          {product.name} - ${product.price}
        </p>
      ))}
    </div>
  );
}

```

17. Common Errors & Fixes

সমস্যা	কারণ	সমাধান
App crashed	start script ভুল	package.json-এ "start": "json-server --watch db.json --host 0.0.0.0 --port \$PORT" আছে কিনা দেখুন
Port error	fixed port ব্যবহার করা হয়েছে	Render-এর জন্য \$PORT ব্যবহার করুন
Cannot find db.json	db.json root folder-এ নেই	db.json root folder-এ রাখুন
API works locally but not on Render	host শুধু localhost	--host 0.0.0.0 ব্যবহার করুন
GitHub push error	branch mismatch	git branch -M main দিয়ে আবার push করুন
First request slow	Render free service sleep করেছে	একটু wait করুন, তারপর refresh করুন
Data reset হয়ে যাচ্ছে	Render filesystem persistent নয়	real database ব্যবহার করুন

সমস্যা	কারণ	সমাধান
Next.js fetch error	wrong API URL	<code>.env.local</code> এবং deployed URL check করুন
CORS error	version/config issue	stable <code>json-server@0.17.4</code> ব্যবহার করুন

18. Final Checklist

Deploy করার আগে check করুন:

- `db.json` root folder-এ আছে
- `package.json`-এ correct `start` script আছে
- `json-server@0.17.4` installed
- `public/.gitkeep` আছে
- Local এ `npm run dev` successful
- GitHub repo public/private properly connected
- Render Build Command: `npm install`
- Render Start Command: `npm start`
- Next.js `.env.local` -এ correct API URL আছে

19. Best Practices

- Practice / assignment / frontend demo-এর জন্য JSON Server ব্যবহার করুন
- Real production app-এর জন্য JSON Server database হিসেবে ব্যবহার করবেন না
- API URL কখনো unnecessarily hardcode করবেন না
- Deploy করার আগে local test করুন
- Free Render service হলে first request slow হতে পারে—এটা normal
- Data permanently save করতে চাইলে real database ব্যবহার করুন

Short Summary

এই setup-এ `db.json` হলো আপনার fake database, JSON Server সেটাকে REST API বানায়, Render সেটাকে live URL দেয়, আর Next.js সেই URL থেকে data fetch করে UI-তে দেখায়।

Final flow:

db.json → JSON Server → Render Live API → Next.js fetch → UI